Homework 08: Set covering, tours, and piecewise-linear costs

ISyE 524 — Fall 2026
Due: Monday, November 9, 2026, at 11:59 p.m. (Madison time).

Complete all three problems. Enumerate delivery routes, model a repeating production sequence using Miller–Tucker–Zemlin (MTZ), and revisit the HW04 distribution network with piecewise-linear shipping discounts. Write each mathematical formulation before implementing it in Julia/JuMP. Use the supplied decision-variable definitions and HiGHS. Explain your reasoning and check solver status and solution availability before reading values.

Submit **one PDF to Gradescope** and mark which pages belong to each problem. **You will not receive full marks unless you mark the pages.**

Getting started and written answers

Follow Assignments and PDF Submission. In VS Code, use **Tasks: Run Task** to run **ISyE 524: Update course repository**, then **ISyE 524: Start an assignment from its template**, entering `hw08`. Open `student-work/hw08/hw08.ipynb`. An existing working copy will not be overwritten.

For **every written part**, type LaTeX mathematics in Markdown or insert a clear photograph/scan of handwritten work. Include the answer and reasoning together and label the subparts; no duplicate transcription is required. Julia/JuMP implementations belong in code cells.

Use `$x_1 \geq 0$` for inline mathematics and `$$x_1+x_2\leq5$$` for display mathematics. Run the Markdown cell to render it.

For handwritten work, edit a Markdown cell in VS Code, drag in a cropped PNG/JPEG, and choose **Insert Image as Attachment**. Alternatively, put the image beside your working notebook under `student-work/hw08/images/` and insert `![My answer](images/problem1.png)`. Keep external images with the notebook. See Images and handwritten work.

Back up `student-work/hw08/` separately; Git does not back up your answers or images. After completing the notebook, export it with the course PDF task, inspect `submissions/hw08.pdf` for readable mathematics, tables, outputs, and images, and submit that PDF to Gradescope.

Computational setup

Use the course Julia environment. All numerical data are supplied below; no separate data files are required. Complete the model and reporting cells. The supplied setup and data cells run without unfinished answers.

AI tools may help debug your own code, explain syntax or error messages, and critique your work. They should not replace constructing your formulation, implementation, or written solution. Complete the required collaboration and LLM-use statement at the end.

```
using JuMP, HiGHS, Printf
```

1: Routing apple deliveries

A local orchard must deliver apples to six customers. The orchard is location 0; customers form the set $I = \{1, \dots, 6\}$. The travel time T_{ab}, in minutes, for a direct leg from location a to location b is given below. Times are the same in both directions. Use these direct-leg times when calculating a route; do not replace a leg by a path through other locations.

From/to	0	1	2	3	4	5	6
0	0	18	9	11	14	21	12
1	18	0	9	16	4	5	10
2	9	9	0	9	6	13	4
3	11	16	9	0	14	21	22
4	14	4	6	14	0	7	9
5	21	5	13	21	7	0	12
6	12	10	4	22	9	12	0

Customer i	1	2	3	4	5	6
Order q_i (crates)	60	55	30	45	25	50

The orchard has one truck with capacity $Q = 100$ crates. It can make several trips, each starting and ending at the orchard. A trip visits each of its customers once, and their total order cannot exceed Q. Each order must be delivered in full on one trip. Ignore loading and unloading times, and assume there is no limit on the total duration of the working day.

1(a): Enumerate feasible routes

List **every nonempty customer subset** whose total order fits on the truck. For each subset, determine a shortest visit order starting and ending at 0. Keep one shortest route per subset; reversed routes have the same time and need not be listed separately. For any subset containing three or more customers, show how you compared the different visit orders.

Give a table with a route index j, customers visited, total crates, the complete visit sequence including 0, and travel time t_j in minutes. You may enumerate by hand or write code; include the resulting table either way.

```
I = 1:6
V = 0:6
Q = 100
q = Dict(zip(I, [60, 55, 30, 45, 25, 50]))
travel = [
     0 18  9 11 14 21 12
    18  0  9 16  4  5 10
     9  9  0  9  6 13  4
    11 16  9  0 14 21 22
    14  4  6 14  0  7  9
    21  5 13 21  7  0 12
    12 10  4 22  9 12  0
]
T = Dict((a, b) => travel[a + 1, b + 1] for a in V, b in V)

# Enter only customer indices, for example route_time([1, 3]).
# The helper includes the departure from and return to the orchard.
function route_time(order)
    path = [0; collect(order); 0]
    return sum(T[path[k], path[k + 1]] for k in 1:length(path)-1)
end;
```

Your route table and reasoning:

[Type your answer and reasoning here, or insert a clear photograph/scan of handwritten work.]

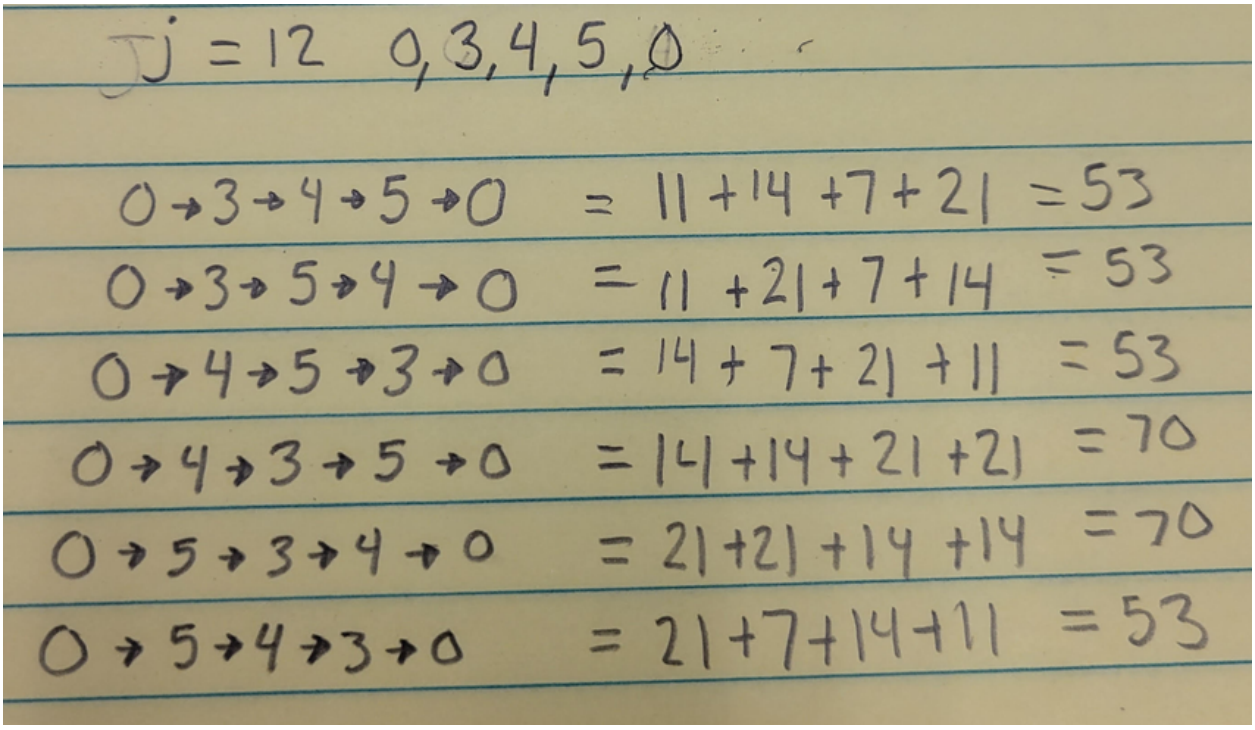


Figure 1: Screenshot From 2026-11-09 08-02-25.png

j	Customers Visited	Total Crates	Delivery Order	t_j (min)
1	1	60	0,1,0	36
2	2	55	0,2,0	18
3	3	30	0,3,0	22
4	4	45	0,4,0	28
5	5	25	0,5,0	42
6	6	50	0,6,0	24
7	1,3	90	0,1,3,0	45
8	1,5	85	0,1,5,0	44
9	2,3	85	0,2,3,0	29
10	2,4	100	0,2,4,0	29
11	2,5	80	0,2,5,0	43
12	3,4,5	100	0,3,4,5,0	53
13	3,4	75	0,3,4,0	39
14	3,5	55	0,3,5,0	53
15	3,6	80	0,3,6,0	45
16	4,5	70	0,4,5,0	42
17	4,6	95	0,4,6,0	35
18	5,6	75	0,5,6,0	45

```
# Optional: enumerate routes and/or construct your route table here.
# Use one entry in routes for each feasible customer subset.
```

```
A = Int[
    1 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0;
    0 1 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0;
    0 0 1 0 0 0 1 0 1 0 0 1 1 1 1 0 0 0;
    0 0 0 1 0 0 0 0 0 1 0 1 1 0 0 1 1 0;
    0 0 0 0 1 0 0 1 0 0 1 1 0 1 0 1 0 1;
    0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 1 1
]


t = [36, 18, 22, 28, 42, 24, 45, 44, 29,
     29, 43, 53, 39, 53, 45, 42, 35, 45]


J = Dict(zip(1:18, [
    [1], [2], [3], [4], [5], [6],
    [1, 3], [1, 5], [2, 3], [2, 4], [2, 5],
    [3, 4, 5], [3, 4], [3, 5], [3, 6],
    [4, 5], [4, 6], [5, 6]
]))
```

```
Dict{Int64, Vector{Int64}} with 18 entries:
  5  => [5]
  16 => [4, 5]
  7  => [1, 3]
  12 => [3, 4, 5]
  8  => [1, 5]
  17 => [4, 6]
  1  => [1]
  4  => [4]
  6  => [6]
  13 => [3, 4]
  2  => [2]
  10 => [2, 4]
  11 => [2, 5]
  9  => [2, 3]
  15 => [3, 6]
  18 => [5, 6]
  14 => [3, 5]
  3  => [3]
```

1(b): Select routes using set covering

Use your route list to define J, the set of route indices, and $A_{ij} = 1$ if route j visits customer i and 0 otherwise. Use the supplied decision variables

$$x_j = \begin{cases} 1 & \text{if route } j \text{ is selected,} \\ 0 & \text{otherwise,} \end{cases} \qquad j \in J.$$

As a first planning model, require each customer to appear on **at least one** selected route. Write a set-covering formulation minimizing total travel time. Capacity and visit-order feasibility should already be encoded in your columns.

Implement your model using `x[j]`. Report the selected complete routes, their loads, and total travel time. Does this solution already visit every customer exactly once? Part 1(c) will enforce that requirement explicitly.

Your formulation and reasoning:

[Type your answer and reasoning here, or insert a clear photograph/scan of handwritten work.]

```
# Construct J, t, and A from your route list if you have not done so.
# Build and solve the set-covering model.
# Report complete routes, loads, total time, and visit counts.

m=Model(HiGHS.Optimizer)

@variable(m,x[1:length(J)], Bin)


@constraint(m, [i in I], sum(A[i, j] * x[j] for j in 1:length(J)) >= 1)


@objective(m, Min, sum(x[j]*t[j] for j in 1:length(J)))

optimize!(m)
if !is_solved_and_feasible(m; allow_local = false)
    error("No optimal primal solution: $(termination_status(m)), $(primal_status
end

println(termination_status(m))

println("minimum time to deliver : ",objective_value(m), " minuites")

route_selected=zeros(18)

for j in 1:length(J)
    route_selected[j]=value(x[j])
end

println(route_selected)
```

```
Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT licenc
```

```
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full de
Using BLAS: blastrampoline
MIP has 6 rows; 18 cols; 31 nonzeros; 18 integer variables (18 binary)
Coefficient ranges:
  Matrix  [1e+00, 1e+00]
  Cost    [2e+01, 5e+01]
  Bound   [1e+00, 1e+00]
  RHS     [1e+00, 1e+00]
Presolving model
6 rows, 18 cols, 31 nonzeros 0s
6 rows, 16 cols, 28 nonzeros 0s
Presolve reductions: rows 6(-0); columns 16(-2); nonzeros 28(-3)
Objective function is integral with scale 1

Solving MIP model with:
   6 rows
   16 cols (16 binary, 0 integer, 0 implied int., 0 continuous, 0 domain fixed)
   28 nonzeros
   Thread count 2 (of 4 threads). Using 1 max workers. Parallel search off

Src: B => Branching; C => Central rounding; F => Feasibility pump; H => Heuristi
     I => Shifting; J => Feasibility jump; L => Sub-MIP; P => Empty MIP; R => Ra
     S => Solve LP; T => Evaluate node; U => Unbounded; X => User solution; Y =>
     Z => ZI Round; l => Trivial lower; p => Trivial point; u => Trivial upper;

        Nodes      |    B&B Tree     |            Objective Bounds
Src  Proc. InQueue |  Leaves   Expl. | BestBound       BestSol              Gap

 u       0       0         0   0.00%   -inf            577                Large
 J       0       0         0   0.00%   -inf            524                Large
 T       0       0         0   0.00%   0               108              100.00%
         1       0         1 100.00%   108             108                0.00%

Solving report
  Status            Optimal
  Primal bound      108
  Dual bound        108
  Gap               0% (tolerance: 0.01%)
  P-D integral      0.000166654586792
  Solution status   feasible
                    108 (objective)
                    0 (bound viol.)
                    0 (int. viol.)
                    0 (row viol.)
  Timing            0.01
                    0.00 (Presolve)
                    0.00 (Solve)
                    0.00 (Postsolve)
```

```
  Max sub-MIP depth 0
  Nodes             1
  Repair LPs        0
  LP iterations     7
                    0 (strong br.)
                    0 (separation)
                    0 (heuristics)
OPTIMAL
minimum time to deliver : 108.0 minuites
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, -0.0, 1.0, 1.0, 0.0, 0.0, -0.0, 0.0, 0.0, -
0.0, 0.0, 1.0, 0.0]
```

Your results and interpretation:

[Type your answer and reasoning here, or insert a clear photograph/scan of handwritten work.]

Route 8,9 and 17 were selected. Route 8 has a load of 85 crates, route 9 has a load of 85 crates and route 17 has a load of 95 crates. The resulting total time is 108 minuites. This solution covers each customer exactly once already.

1(c): Enforce exactly one delivery

Modify the formulation so that each customer appears on **exactly one** selected trip. Write the changed constraints and solve the revised model. Report only its selected routes and total travel time, and compare with 1(b). Explain why a covering constraint does not generally enforce exactly-once service, even if it happens to do so in your solution to 1(b).

Your changed formulation and explanation:

[Type your answer and reasoning here, or insert a clear photograph/scan of handwritten work.]

```julia
# Solve the exactly-once version and report its routes and total time.
m2=Model(HiGHS.Optimizer)

@variable(m2,x[1:length(J)], Bin)


@constraint(m2, [i in I], sum(A[i, j] * x[j] for j in 1:length(J)) == 1)


@objective(m2, Min, sum(x[j]*t[j] for j in 1:length(J)))

optimize!(m2)
if !is_solved_and_feasible(m; allow_local = false)
    error("No optimal primal solution: $(termination_status(m2)), $(primal_statu
end

println(termination_status(m2))
```

```julia
println("minimum time to deliver : ",objective_value(m2), " minuites")

route_selected2=zeros(18)

for j in 1:length(J)
    route_selected2[j]=value(x[j])
end

println(route_selected)
```

```
Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT licenc
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full de
Using BLAS: blastrampoline
MIP has 6 rows; 18 cols; 31 nonzeros; 18 integer variables (18 binary)
Coefficient ranges:
  Matrix  [1e+00, 1e+00]
  Cost    [2e+01, 5e+01]
  Bound   [1e+00, 1e+00]
  RHS     [1e+00, 1e+00]
Presolving model
6 rows, 18 cols, 31 nonzeros 0s
6 rows, 18 cols, 31 nonzeros 0s
Presolve reductions: rows 6(-0); columns 18(-0); nonzeros 31(-0) - Not reduced
Objective function is integral with scale 1

Solving MIP model with:
   6 rows
   18 cols (18 binary, 0 integer, 0 implied int., 0 continuous, 0 domain fixed)
   31 nonzeros
   Thread count 2 (of 4 threads). Using 1 max workers. Parallel search off

Src: B => Branching; C => Central rounding; F => Feasibility pump; H => Heuristi
     I => Shifting; J => Feasibility jump; L => Sub-MIP; P => Empty MIP; R => Ra
     S => Solve LP; T => Evaluate node; U => Unbounded; X => User solution; Y =>
     Z => ZI Round; l => Trivial lower; p => Trivial point; u => Trivial upper;

        Nodes      |    B&B Tree     |            Objective Bounds
Src  Proc. InQueue |  Leaves   Expl. | BestBound       BestSol              Gap

 J       0       0         0   0.00%   -inf            131                Large
 T       0       0         0   0.00%   0               108              100.00%
         1       0         1 100.00%   108             108                0.00%

Solving report
  Status            Optimal
  Primal bound      108
```

```
  Dual bound        108
  Gap               0% (tolerance: 0.01%)
  P-D integral      0.000243663787842
  Solution status   feasible
                    108 (objective)
                    0 (bound viol.)
                    0 (int. viol.)
                    0 (row viol.)
  Timing            0.01
                    0.00 (Presolve)
                    0.01 (Solve)
                    0.00 (Postsolve)
  Max sub-MIP depth 0
  Nodes             1
  Repair LPs        0
  LP iterations     7
                    0 (strong br.)
                    0 (separation)
                    0 (heuristics)
OPTIMAL
minimum time to deliver : 108.0 minuites
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, -0.0, 1.0, 1.0, 0.0, 0.0, -0.0, 0.0, 0.0, -
0.0, 0.0, 1.0, 0.0]
```

Your comparison:

[Type your answer and reasoning here, or insert a clear photograph/scan of handwritten work.]

The selected routes are 8,9, and 17 with a total travel time of 108 minuites. This matches the covering result which allows for overlap to occur. It is unlikley for these two results to match because set partitioning only allows for each element to be used exactly once. The only way these two match is if there happens to be a combiation of sets that fully covers everthing and is also the minimum cost/ satifys the objective function.

2: Paint production as a traveling salesperson problem

A company produces the same five batches of paint each week using one blender. Batch $i \in J = \{1, \ldots, 5\}$ takes b_i minutes to blend. Cleaning between consecutive batches depends on their order: A_{ij} is the number of minutes needed to clean after batch i if batch j is next.

After i / before j	1	2	3	4	5
1	0	11	7	13	11
2	5	0	13	15	15
3	13	15	0	23	11
4	9	13	5	0	3
5	3	7	7	7	0

Batch i	1	2	3	4	5
Blending time b_i (minutes)	40	35	45	32	50

Every batch must be produced once per week. The sequence repeats each week, so include cleaning between the **last batch of one week and the first batch of the next**. Choose a cyclic production sequence minimizing total blending and cleaning time per week. Rotating a cycle does not change its cost, so label batch 1 as the first batch.

Use these supplied decision variables, with $n = |J| = 5$:

- $x_{ij} \in \{0, 1\}$: 1 if batch j immediately follows batch i in the repeating cycle, and 0 otherwise, for $i, j \in J$.
- $u_i \in \mathbb{R}$, $1 \leq u_i \leq n$: relative position of batch i in the cycle, for $i \in J$; fix $u_1 = 1$.

The u_i are continuous; only the x_{ij} need to be binary.

2(a): Formulate using MTZ

Write a mixed-integer linear formulation using **Miller–Tucker–Zemlin** constraints. Include the objective, one-predecessor and one-successor constraints, no self-loops, variable domains, and the MTZ conditions.

Begin with the implication that choosing arc (i, j) forces the position to increase by at least 1. Explain which arcs are exempt and why. Derive a valid relaxation constant from the supplied bounds on u. Explain whether blending times affect the best sequence.

Your formulation and reasoning:

[Type your answer and reasoning here, or insert a clear photograph/scan of handwritten work.]

```
J = 1:5
n = length(J)
A = [
     0 11  7 13 11
     5  0 13 15 15
    13 15  0 23 11
     9 13  5  0  3
     3  7  7  7  0
]
b = [40, 35, 45, 32, 50];
```

2(b): Solve and interpret

Implement your MTZ formulation using `x[i, j]` and `u[i]`. Report a complete cycle beginning and ending at batch 1, the total cleaning time, the total blending time, and the total time per week. Include the last-to-first cleaning transition in your calculation.

```
# Build and solve the MTZ model.
# Report the ordered cycle and cleaning, blending, and total minutes.
```

Your results and interpretation:

[Type your answer and reasoning here, or insert a clear photograph/scan of handwritten work.]

2(c): What goes wrong without MTZ?

Remove the position variables and MTZ constraints, retaining binary arc variables, one predecessor, one successor, and no self-loops. Solve this model and report its directed cycles and cleaning time.

Explain why that solution is not a valid repeating production sequence. For one subtour that does not contain batch 1, add together the MTZ inequalities around that subtour and show why they forbid it.

```
# Solve the model without MTZ.
# Report its directed cycles and cleaning time.
```

Your results and explanation:

[Type your answer and reasoning here, or insert a clear photograph/scan of handwritten work.]

3: Revisiting HW04 — piecewise-linear shipping costs

You studied this same distribution network in HW04, Problem 2. There, shipping costs were linear in the amount shipped. Reuse its flow-balance model and extend it with lecture 17's **multiple-choice piecewise-linear formulation** to represent incremental shipping discounts. All network data and one optimal HW04 shipment plan are repeated here.

A distributor moves one interchangeable product through ten locations. Locations 1, 2, and 9 supply product; locations 6, 7, and 8 require it. Other locations are transfer points. Product may also pass through a location with supply or demand. Positive net supply b_i means supply, negative means demand, and zero means pure transshipment.

Node i	1	2	3	4	5	6	7	8	9	10
b_i (tons)	10	15	0	0	0	-12	-11	-7	5	0

Only the sixteen directed arcs in the following table are available. An arc does not imply that its reverse exists. All supply must be shipped, all demand must be met exactly, and there is no loss in transit. Quantities are **continuous tons**, as in HW04.

The table retains HW04's rate c_{ij} and capacity u_{ij}. The last column gives one shipment plan f^{HW04} that was optimal under the old linear costs, with total cost **$242**. It is supplied for the comparison in part 3(c).

From	To	c_{ij} (\$/ton)	u_{ij} (tons)	f_{ij}^{HW04} (tons)
1	3	5	15	10
2	4	4	20	15
3	4	2	12	0
3	5	6	10	0
3	6	5	10	10
4	5	1	10	8
4	8	2	10	7
5	3	4	8	0
5	6	6	10	2
5	7	3	15	11
8	7	4	8	0
9	3	3	5	0
9	10	2	5	5
10	5	1	6	5
10	8	4	4	0
3	10	2	6	0

New incremental tariff

The carrier now offers the following rates on **each arc separately**:

Portion of an arc's shipment	Marginal rate (\$/ton)
First 2 tons	c_{ij}
Next 2 tons (above 2 and through 4 tons)	$0.75c_{ij}$
Any amount above 4 tons, up to capacity u_{ij}	$0.50c_{ij}$

The discounts are **incremental**: the lower rate applies only to the portion in that tier, not to all tons shipped on the arc. For example, a 6-ton shipment is charged for 2 tons at each of the three rates. Each arc's total shipment during the planning period determines its bill; splitting it into bookkeeping entries does not reset the tiers. There is no fixed charge and a zero shipment costs zero. All arc capacities are at least 4 tons; an arc with capacity exactly 4 tons has no positive-length third tier.

Let N and A be the node and arc sets. Use these supplied variables:

- $f_{ij} \in [0, u_{ij}]$: continuous tons shipped on arc $(i, j) \in A$.
- $t_{ij} \geq 0$: total shipping cost on arc $(i, j) \in A$, in dollars.
- $z_{ijk} \in \{0, 1\}$: selects quantity interval $k \in K = \{1, 2, 3\}$ on arc $(i, j) \in A$; exactly one interval is selected per arc.
- $w_{ijk} \geq 0$: continuous quantity, in tons, equal to f_{ij} in the selected interval and zero in all other intervals. It represents the **whole arc shipment** in that interval, not just the tons charged at that tier's marginal rate.

The interval endpoints are $B_{ij,0} = 0$, $B_{ij,1} = 2$, $B_{ij,2} = 4$, and $B_{ij,3} = u_{ij}$.

3(a): Derive the piecewise-linear cost

Write the total arc cost $C_{ij}(f)$ in slope-intercept form on the intervals $[0, 2]$, $[2, 4]$, and $[4, u_{ij}]$. Express the slopes and intercepts in terms of c_{ij}, and check that adjacent pieces agree at 2 and 4 tons.

Is C_{ij} convex, concave, both, or neither? Explain why simply minimizing a linear expression with an independent quantity for each price tier and upper bounds on the first two tiers would permit an incorrect use of the discounts without additional linking constraints.

Your cost function and reasoning:

[Type your answer and reasoning here, or insert a clear photograph/scan of handwritten work.]

```
N = 1:10
b = Dict(zip(N, [10, 15, 0, 0, 0, -12, -11, -7, 5, 0]))
# Each row is (from, to, cost, capacity).
arc_data = [
    (1, 3, 5, 15), (2, 4, 4, 20), (3, 4, 2, 12),
    (3, 5, 6, 10), (3, 6, 5, 10), (4, 5, 1, 10),
    (4, 8, 2, 10), (5, 3, 4, 8), (5, 6, 6, 10),
    (5, 7, 3, 15), (8, 7, 4, 8), (9, 3, 3, 5),
    (9, 10, 2, 5), (10, 5, 1, 6), (10, 8, 4, 4),
    (3, 10, 2, 6)
]
A = [(i, j) for (i, j, cost, capacity) in arc_data]
c = Dict((i, j) => cost for (i, j, cost, capacity) in arc_data)
u = Dict((i, j) => capacity for (i, j, cost, capacity) in arc_data)

# Same network data as HW04, Problem 2.
# Intervals are indexed by k=1,2,3; endpoints by k=0,1,2,3.
K = 1:3
B = Dict((a, k) => (k == 3 ? u[a] : 2.0 * k)
         for a in A, k in 0:3)
f_HW04 = Dict(zip(A, [10, 15, 0, 0, 10, 8, 7, 0,
                      2, 11, 0, 0, 5, 5, 0, 0]));
```

3(b): Extend the HW04 model and solve

Write a complete MILP minimizing total shipping cost under the new tariffs. Retain HW04's outflow-minus-inflow balances and arc capacity limits. Use lecture 17's **multiple-choice formulation**, with z_{ijk} selecting an interval, w_{ijk} representing its shipment quantity, and t_{ij} equal to its correctly accumulated cost. Include every variable domain.

Implement and solve the formulation in Julia/JuMP. You may adapt your HW04 network code. For tuple $a = (i, j)$, use `f[a]`, `t[a]`, `z[a,k]`, and `w[a,k]`, matching the supplied `A`, `c`, `u`, and `B` dictionaries. Keep f and w continuous; the interval selectors are the binary variables.

Report the minimum total cost and every positive arc flow in tons.

Your MILP formulation and reasoning:

[Type your answer and reasoning here, or insert a clear photograph/scan of handwritten work.]

```
# Extend the HW04 flow model with multiple-choice cost constraints.
# Solve and report the minimum cost and all positive flows.
```

Your optimal shipment plan:

[Type your answer and reasoning here, or insert a clear photograph/scan of handwritten work.]

3(c): Compare with the earlier HW04 plan

Evaluate the supplied f^{HW04} under the **new incremental tariffs**. Report its new cost and compare it with your optimum from 3(b). Can the earlier plan still be used optimally, or does reoptimizing improve it?

Using the original \$242 linear-cost benchmark, separate the savings into (i) savings from applying the discounts to the supplied old plan and (ii) any additional savings from reoptimizing under those discounts. Explain what the comparison shows. No additional optimization solve is required.

```
# Price f_HW04 under the new tariffs and compare with your new optimum.
# Separate the discount savings from any additional reoptimization savings.
```

Your comparison and explanation:

[Type your answer and reasoning here, or insert a clear photograph/scan of handwritten work.]

Collaboration and LLM-use statement — required

Complete this statement as the **very last part of your homework**. Be specific enough that a reader can understand what work you did yourself and how others or LLMs helped you learn.

- **People:** Give the name of every person you collaborated with, identify the problem/subpart you discussed, and explain the nature of the collaboration. If you worked alone, explicitly state that you did not collaborate with anyone.
- **LLMs/tools:** Name every LLM or AI assistant you used and identify the model/version as precisely as available. If the interface did not show a model/version, say so. If you used none, explicitly state that you did not use an LLM or AI assistant.
- **Specific assistance and learning:** For each tool, identify the problem/subpart, describe what you asked it to help with, what suggestions you used or rejected, and how the interaction helped you understand the mathematics or implementation. Explain how you checked the suggestions. "I used ChatGPT for help" is not sufficiently specific.

Disclosure does not replace the requirement that your submitted formulation, implementation, and explanation be your own work and that you understand them.

People and collaboration:

[Names, problem/subparts, and what you discussed; or explicitly state that you worked alone.]

I worked alone

LLMs/tools and models:

[Tool names and models/versions, or explicitly state that you used none.]

Gemini 3.6

How I used the assistance, what I learned, and how I checked it:

[Specific details for each tool and relevant problem/subpart; state not applicable if you used none.]

I used it to debug error messages when I used len instead of length on the first problem. len is the same as length just the python version and I had it mixed up.